

briefly look at TLS 1.2 as it is the most stable and widely deployed security protocol till date.

Let's begin by running through the HTTP 1.1 protocol that was covered in the earlier article in this series. HTTP 1.1 follows a simple text-based request/response message format, as shown in the sample transaction below.

HTTP request - sent by client

```
GET / HTTP/1.1
Host: www.example.com
User-Agent: curl/7.65.0
Accept: */*
```

HTTP response - sent by server

```
HTTP/1.1 200 OK
Accept-Ranges: bytes
Cache-Control: max-age=604800
Content-Type: text/html; charset=UTF-8
Date: Sat, 11 Jan 2020 06:30:35 GMT
Etag: "3147526947+ident"
Expires: Sat, 18 Jan 2020 06:30:35 GMT
Last-Modified: Thu, 17 Oct 2019 07:18:26 GMT
Server: ECS (sjc/4E73)
Vary: Accept-Encoding
X-Cache: HIT
Content-Length: 1256
```

```
<!doctype html>
<html>
<head>

<title>Example Domain</title>

<meta charset="utf-8" />
...
```

The key features of HTTP 1.1 are:

1. Message headers and metadata are sent in text format.
2. Limited message concurrency:
 - a. In HTTP 1.0, only one message could be sent on a connection. This imposed a connection setup overhead for every transaction and was quickly seen as inefficient.
 - b. HTTP 1.1 improved on this with persistent connections,

i.e., a client could send multiple requests on the same connection.

- c. HTTP 1.1 also introduced request pipelining. With this, a client did not have to wait till a response for an earlier request was received before it sent the next request. However, the server still processed one request at a time and in the order in which these were received.

HTTP/2: An overview

As seen earlier, the limited concurrency of HTTP 1.1 causes the browser to wait till earlier requests were responded to, even when the server could have satisfied the requests simultaneously. HTTP/2 introduced the concept of a stream of data that roughly corresponds to the request/response pair of HTTP 1.1. However, HTTP/2 goes beyond that, and permits simultaneous multi-stream communication on the same connection, thereby removing the limit on concurrency.

The other limiting factor in HTTP 1.1 was its extensive use of text in HTTP metadata (i.e., headers). Information can be represented in a more compact manner in binary format as compared to text. This is yet another enhancement driven by HTTP/2, where headers are compressed and transmitted in binary form.

There are also cases when once the server delivers certain content, there is a high chance that the client will ask for certain other linked content. At such times, it is an obvious performance benefit if the server can 'push' the next level of content, even before the client requests for it. HTTP/2 provides the 'server push' feature to cater for such scenarios.

Multiple streams: HTTP/2 introduces a new paradigm within HTTP by taking the original request/response and making it flexible in different ways. These modified request/

response units are called streams, which have the following characteristics:

1. Like in HTTP 1.1, multiple streams can be transferred on a single connection.
2. Multiple streams can be in progress, simultaneously.
3. Additionally, streams can be initiated even by the server, while in HTTP 1.1, the server never initiated communication.

When multiple streams are in progress at the same time, the sender must indicate which stream the data being sent belongs to. This leads to the concept of frames, which are the basic units of communication in HTTP/2. When an endpoint must send data, it selects the stream to which the data belongs, forms a frame, tags it with the stream-ID, and then sends it over the connection. The stream-ID is an integer that identifies the stream.

Streams are considered opened when they are first used by an endpoint. When an endpoint opens a stream, it needs to pick a stream-ID. This has been simplified with the following two rules: a) clients use odd integers and servers use even integers; and b) within a client or server, a hitherto unused number is used for a new stream. Thus, the first client stream typically has the ID of 1, the next one may have the ID 3 and so on, while the first server stream-ID is typically 2, the next is 4, and so on.

Stream-ID 0 is special and deserves some discussion. With the concurrent, multi-stream model of HTTP 2.0, it is possible that one of the endpoints is generating huge amounts of data that the other endpoint cannot process fast enough. In that case, the peer endpoint can limit the inflow of data by reducing a window (akin to TCP windowing). Such information is communicated over stream-ID 0.

Server push: The following from RFC 7540 ('Requests for Comments' or RFCs are articles on tech topics published by the Internet Engineering